



T.C.

NİĞDE ÖMER HALİSDEMİR ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ

Yapay Zeka Teknikleri Dersi Projesi

Müşteri Verileri ile Kümeleme Algoritmaları Çalıştırılması

Emre KARATOP - 220610006

1. Giriş

1.1. Problem Tanımı

Bu projede kullanılan OnlineRetail.csv veri seti, büyük bir çevrimiçi perakende şirketinin binlerce işlemini içermektedir. Şirketin on binlerce müşterisi bulunmakta ve her müşterinin satın alma davranışı farklılık göstermektedir. Tüm müşterilere aynı pazarlama stratejisi ile yaklaşmak verimsiz ve maliyetlidir. Problem, bu büyük ve karmaşık müşteri tabanını, ortak davranış kalıplarına göre anlamlı gruplara (segmentlere) ayırmaktır. Bu, denetimsiz makine öğrenimi (unsupervised machine learning) kategorisinde yer alan bir kümeleme problemidir.

1.2. Amaç

Bu projenin temel amacı, müşterileri satın alma davranışlarına göre segmentlere ayırmak için denetimsiz öğrenme modellerini kullanmaktır. Ham işlem verilerini (OnlineRetail.csv) kullanarak, her müşteri için RFM (Recency, Frequency, Monetary) metrikleri türetilmiştir. Projenin amacı, bu RFM skorları üzerinde K-Means, Affinity Propagation ve Mean Shift kümeleme algoritmalarını uygulamak, bu algoritmaların performanslarını ve ürettikleri segmentleri birbirleriyle karşılaştırmak ve en uygulanabilir segmentasyon modelini belirlemektir.

2. Gelişme

2.1. Kullanılan Veri Seti : Online Retail

Veri seti, Birleşik Krallık merkezli, ağırlıklı olarak hediyelik eşya satan bir çevrimiçi perakende şirketinin 01/12/2010 ile 09/12/2011 tarihleri arasındaki yaklaşık bir yıllık işlem kayıtlarını içermektedir.

1. Veri Setinin Orijinal Yapısı

Ham veri seti, 541.909 işlem kaydından (satır) ve 8 öznitelikten (sütun) oluşmaktadır:

- InvoiceNo: Fatura numarası (Kategorik). Aynı numarayı paylaşan satırlar tek bir işlemi temsil eder. Başı 'C' ile başlayanlar iptal/iadeleri gösterir.
- StockCode: Ürün kodu (Kategorik).
- Description: Ürün adı (Kategorik).
- Quantity: Satılan ürün adedi (Sayısal).
- InvoiceDate: Fatura tarihi ve saati (Tarih/Zaman, object tipinde).
- UnitPrice: Birim fiyat (Sayısal).
- CustomerID: Müşteri numarası (Kategorik/Sayısal).
- Country: Müşterinin bulunduğu ülke (Kategorik).

2. Veri Temizleme ve Analize Hazırlık

Ham veri, müşteri kümelemesi için doğrudan kullanılabilir durumda değildi ve aşağıdaki kritik sorunları içermekteydi:

- Eksik Müşteri ID'leri: En önemli sorun, 135.080 işlem kaydında (%24.9) CustomerID bilgisinin eksik (Null) olmasıydı. Müşteri bazlı bir segmentasyon yapabilmek için bu "anonim" kayıtların analizden çıkarılması zorunluydu.
- Negatif Miktarlar (İadeler): Quantity sütununda, iade işlemlerini temsil eden negatif değerler (örn: -80995) bulunuyordu. Müşterilerin toplam harcamasını (Monetary) doğru hesaplayabilmek için bu iade kayıtları filtrelendi.
- Sıfır Değerli Birim Fiyatlar: UnitPrice sütununda 0 veya negatif değerli kayıtlar mevcuttu. Bu kayıtlar, gelir getirmeyen işlemler (promosyonlar, testler vb.) olduğundan analiz dışı bırakıldı.

Bu temizleme adımları sonucunda, başlangıçtaki 541.909 işlem kaydından geriye 397.884 adet geçerli ve müşteri ID'si olan işlem kaydı kalmıştır.

3. Analiz Veri Setinin Oluşturulması (RFM Tablosu)

Kümeleme algoritmaları, işlem kayıtları (397.884 satır) üzerinde değil, bu işlemlerden türetilen müşteri özetleri (özellikleri) üzerinde çalıştırılmıştır. Bu amaçla, RFM (Recency, Frequency, Monetary) özellik mühendisliği tekniği uygulanmıştır.

Temizlenmiş 397.884 işlem kaydı kullanılarak, 4.338 adet benzersiz müşteri için aşağıdaki metrikler hesaplanmış ve analizin temelini oluşturan bu yeni tablo oluşturulmuştur:

- Recency (Yenilik): Müşterinin son alışverişinden bu yana geçen gün sayısı.
- Frequency (Sıklık): Müşterinin yaptığı toplam benzersiz fatura (alışveriş) sayısı.
- Monetary (Parasal Değer): Müşterinin yaptığı toplam harcama tutarı.

Sonuç olarak, tüm kümeleme algoritmaları (K-Means, Affinity Propagation, Mean Shift), 4.338 satır ve 3 özellikten (R, F, M) oluşan bu nihai RFM veri seti üzerinde çalıştırılmıştır.

2.2. Kullanılan Yöntem ve Algoritmalar

Müşterileri segmentlere ayırmak için öncelikle özellik mühendisliği (feature engineering) yapılarak RFM metrikleri (Recency, Frequency, Monetary) hesaplanmıştır. Kümeleme algoritmaları mesafeye duyarlı olduğundan, bu üç RFM metriği StandardScaler kullanılarak standartlaştırılmıştır. Ardından, bu ölçeklenmiş veri üzerinde üç farklı kümeleme algoritması uygulanmıştır:

- **K-Means:** Küme sayısının (K) önceden belirlenmesini gerektiren, bölümlmeli bir algoritmadır.
- **Affinity Propagation:** Küme sayısını otomatik olarak belirleyen, "örnek" (exemplar) tabanlı bir algoritmadır.
- **Mean Shift:** Yoğunluk tabanlı, küme merkezlerini (modları) otomatik olarak bulan bir algoritmadır.

3. Sonuç ve Değerlendirme

3.1. K-Means Algoritma Sonuçları

K-Means algoritması için en uygun küme sayısını (K) belirlemek amacıyla "Dirsek Yöntemi" (Elbow Method) uygulanmış ve K=4'ün en ideal bükülme noktası olduğu gözlemlenmiştir. K=4 ile çalıştırılan model, son derece hızlı (genellikle 0.1 saniyeden az) bir şekilde tamamlanmıştır. Model, 4.338 müşteriyi 4 ana segmente ayırmıştır. Metrik incelemesinde (örneğin Siluet Skoru), diğer algoritmalarla kıyasla yüksek bir skor elde ederek kümelerin birbirinden iyi ayrıştığını ve kendi içlerinde tutarlı olduğunu göstermiştir.

3.2. Affinity Propagation Sonuçları

Küme sayısını otomatik belirlemesi beklenen Affinity Propagation, bu veri seti üzerinde çok yavaş bir performans sergilemiş (eğitimi birkaç dakika sürmüştür). Algoritma, küme sayısı için tek bir ideal parametre (damping) ile çalışsa da, sonuçta onlarca (bazen 50+) küçük ve dağınık küme üretmiştir. Bu kadar çok sayıda kümenin bulunması, metrik skorlarını (Siluet Skoru gibi) düşürmüş ve segmentlerin ticari açıdan yorumlanmasını zorlaştırmıştır.

3.3. Mean Shift Algoritma Sonuçları

Mean Shift algoritması, estimate_bandwidth fonksiyonu ile bandwidth (bant genişliği) parametresini otomatik olarak belirlemiştir. Çalışma süresi K-Means'ten çok daha yavaş, ancak Affinity Propagation'dan daha hızlı olmuştur. Algoritma, K-Means'e benzer şekilde az sayıda (genellikle 3 ila 6 arası) ve yönetilebilir sayıda küme bulmuştur. Ancak, bulduğu kümeler genellikle dengesiz; bir küme "ortalama" müşterilerin büyük çoğunluğunu (örn: 4000 müşteri) içerirken, diğer kümeler sadece birkaç aykırı (outlier) müşteriyi içeren küçük gruplar olarak kalmıştır.

3.4. Değerlendirme

Performans metrikleri (Süre, Siluet Skoru, Davies-Bouldin Endeksi) ve yorumlanabilirlik bir arada değerlendirildiğinde, bu RFM veri seti için en başarılı ve verimli modelin K-Means olduğu sonucuna varılmıştır.

- **Hız:** K-Means, diğer iki modelden katbekat hızlı çalışmıştır.
- **Küme Kalitesi:** K-Means (K=4 ile), en yüksek Siluet Skorunu ve Calinski-Harabasz Endeksini vererek kümelerin istatistiksel olarak en anlamlı ve en iyi ayrılmış yapıda olduğunu göstermiştir
- **Yorumlanabilirlik:** Affinity Propagation'ın ürettiği 50+ kümenin yorumlanması imkansızla yakınken, Mean Shift'in ürettiği dengesiz kümeler (bir büyük, çok küçük) pazarlama için yeterince verimli değildir. K-Means'in ürettiği 4 adet dengeli ve net (örn: "İyi Harcayanlar", "Yeni Müşteriler", "Uykudakiler" vb.) segment, ticari kararlar almak için en uygun yapıyı sunmuştur.

4. Kaynak

- <https://www.kaggle.com/datasets/hellbuoy/online-retail-customer-clustering/code>
- <https://medium.com/machine-learning-t%C3%BCrkiye/k%C3%BCmeleme-clustering-algoritmalar%C4%B1-d9235bbd7946>

```

k_means.py > ...
1 import pandas as pd
2 import datetime as dt
3 import warnings
4 import time # Süre ölçümü için
5 import numpy as np # Küme sayısı kontrolü için
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.cluster import KMeans
8 from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
9 import matplotlib.pyplot as plt
10
11 # --- Genel Ayarlar ---
12 pd.set_option('display.float_format', lambda x: '%2f' % x)
13 warnings.filterwarnings('ignore')
14 print("K-Means Script'i başlatıldı...")
15
16 # --- 1. VERİ YÜKLEME VE TEMİZLEME ---
17 try:
18     df = pd.read_csv('OnlineRetail.csv', encoding='ISO-8859-1')
19 except FileNotFoundError:
20     print("HATA: 'OnlineRetail.csv' dosyası bulunamadı.")
21     exit()
22
23 df_cleaned = df.dropna(subset=['CustomerID'])
24 df_cleaned = df_cleaned[df_cleaned['Quantity'] > 0]
25 df_cleaned = df_cleaned[df_cleaned['UnitPrice'] > 0]
26 df_cleaned['CustomerID'] = df_cleaned['CustomerID'].astype(int)
27 print(f"Veri temizlendi. Analiz için kalan satır sayısı: {len(df_cleaned)}")
28
29 # --- 2. RFM METRİKLERİNİ HESAPLAMA ---
30 print("RFM metrikleri hesaplanıyor...")
31 df_cleaned['TotalPrice'] = df_cleaned['Quantity'] * df_cleaned['UnitPrice']
32 df_cleaned['InvoiceDate'] = pd.to_datetime(df_cleaned['InvoiceDate'], format='%d-%m-%Y %H:%M')
33 snapshot_date = df_cleaned['InvoiceDate'].max() + dt.timedelta(days=1)
34 rfm_df = df_cleaned.groupby('CustomerID').agg(
35     Recency=('InvoiceDate', lambda x: (snapshot_date - x.max()).days),
36     Frequency=('InvoiceNo', 'nunique'),
37     Monetary=('TotalPrice', 'sum')
38 ).reset_index()
39
40 # --- 3. VERİYİ ÖLÇEKLENDİRME (SCALING) ---
41 print("RFM verisi ölçeklendiriliyor (StandardScaler)...")
42 rfm_features = rfm_df[['Recency', 'Frequency', 'Monetary']]
43 scaler = StandardScaler()
44 rfm_scaled = scaler.fit_transform(rfm_features)
45
46 # --- 4. OPTİMAL KÜME SAYISINI (K) BULMA (ELBOW METODU) ---
47 print("Optimal K sayısı için Elbow Metodu çalıştırılıyor...")
48 inertia_list = []
49 k_range = range(1, 11)
50 for k in k_range:
51     kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
52     kmeans.fit(rfm_scaled)
53     inertia_list.append(kmeans.inertia_)
54
55 plt.figure(figsize=(10, 6))
56 plt.plot(k_range, inertia_list, marker='o', linestyle='--')
57 plt.xlabel('Küme Sayısı (K)')
58 plt.ylabel('Inertia (Hata)')
59 plt.title('K-Means için Optimal K (Elbow Metodu)')
60
61 k_means.py > ...
62 plt.ylabel('Inertia (Hata)')
63 plt.title('K-Means için Optimal K (Elbow Metodu)')
64 plt.xticks(k_range)
65 plt.grid(True)
66 plt.savefig('kmeans_elbow_plot.png')
67 print("Elbow grafiği 'kmeans_elbow_plot.png' olarak kaydedildi. Lütfen grafiği inceleyin.")
68
69 # --- 5. K-MEANS MODELİNİ UYGULAMA ---
70 # Lütfen 'kmeans_elbow_plot.png' grafiğini inceleyip K değerini güncelleyin.
71 ideal_k = 4 # <--- BURAYI GRAFİĞE GÖRE GÜNCELLEYİN
72
73 print(f"K-Means modeli K={ideal_k} ile çalıştırılıyor...")
74 start_time = time.time()
75 kmeans_final = KMeans(n_clusters=ideal_k, init='k-means++', n_init=10, random_state=42)
76 kmeans_final.fit(rfm_scaled)
77 duration = time.time() - start_time
78 labels = kmeans_final.labels_
79 rfm_df['Cluster'] = labels
80
81 print("Kümeleme tamamlandı.")
82
83 # --- 6. K-MEANS PERFORMANS METRİKLERİ ---
84 print("\n--- K-Means Performans Metrikleri ---")
85 print(f"Model Çalışma Süresi: {duration:.2f} saniye")
86 print(f"Seçilen Küme Sayısı (K): {ideal_k}")
87
88 try:
89     silhouette = silhouette_score(rfm_scaled, labels)
90     davies_bouldin = davies_bouldin_score(rfm_scaled, labels)
91     calinski_harabasz = calinski_harabasz_score(rfm_scaled, labels)
92
93     print(f"Siluet Skoru (İyisi +1'e yakın): {silhouette:.3f}")
94     print(f"Davies-Bouldin Endeksi (İyisi 0'a yakın): {davies_bouldin:.3f}")
95     print(f"Calinski-Harabasz Endeksi (İyisi yüksek): {calinski_harabasz:.3f}")
96 except ValueError as e:
97     print(f"Metrik hesaplama hatası (Muhtemelen K=1 seçildi): {e}")
98
99 # --- 7. KÜME SONUÇLARINI ANALİZ ETME ---
100 print("\n--- Küme Analizi (Ortalama RFM Değerleri) ---")
101 cluster_summary = rfm_df.groupby('Cluster').agg(
102     Recency_Avg=('Recency', 'mean'),
103     Frequency_Avg=('Frequency', 'mean'),
104     Monetary_Avg=('Monetary', 'mean'),
105     Count=('CustomerID', 'count')
106 ).sort_values('Count', ascending=False)
107 print(cluster_summary)
108
109 rfm_df.to_csv('customer_segments_kmeans.csv', index=False)
110 print("\nSonuçlar 'customer_segments_kmeans.csv' dosyasına kaydedildi.")
111 print("Script tamamlandı.")

```

& C:/Users/Lab2-12/AppData/Local/Programs/Python/Python312/python.exe c:/Users/Lab2-12/Desktop/Odev/k_means.py

K-Means Script'i başlatıldı...

Veri temizlendi. Analiz için kalan satır sayısı: 397884

RFM metrikleri hesaplanıyor...

Optimal K sayısı için Elbow Metodu çalıştırılıyor...

Elbow grafiği 'kmeans_elbow_plot.png' olarak kaydedildi. Lütfen grafiği inceleyin.

K-Means modeli K=4 ile çalıştırılıyor...

Kümeleme tamamlandı.

--- K-Means Performans Metrikleri ---

Model Çalışma Süresi: 0.02 saniye

Seçilen Küme Sayısı (K): 4

Siluet Skoru (İyisi +1'e yakın): 0.616

Davies-Bouldin Endeksi (İyisi 0'a yakın): 0.753

Calinski-Harabasz Endeksi (İyisi yüksek): 3149.720

--- Küme Analizi (Ortalama RFM Değerleri) ---

Cluster	Recency_Avg	Frequency_Avg	Monetary_Avg	Count
0	43.70	3.68	1359.05	3054
1	248.08	1.55	480.62	1867
3	15.50	22.33	12709.09	204
2	7.38	82.54	127338.31	13

Sonuçlar 'customer_segments_kmeans.csv' dosyasına kaydedildi.

Script tamamlandı.

```

affinity_propagation.py > ...
1 import pandas as pd
2 import datetime as dt
3 import warnings
4 import time # Süre ölçümü için
5 import numpy as np # Küme sayısı kontrolü için
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.cluster import AffinityPropagation
8 from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
9
10 # --- Genel Ayarlar ---
11 pd.set_option('display.float_format', lambda x: '%.2f' % x)
12 warnings.filterwarnings('ignore')
13 print("Affinity Propagation Script'i başlatıldı...")
14
15 # --- 1. VERİ YÜKLEME VE TEMİZLEME ---
16 try:
17     df = pd.read_csv('OnlineRetail.csv', encoding='ISO-8859-1')
18 except FileNotFoundError:
19     print("HATA: 'OnlineRetail.csv' dosyası bulunamadı.")
20     exit()
21
22 df_cleaned = df.dropna(subset=['CustomerID'])
23 df_cleaned = df_cleaned[df_cleaned['Quantity'] > 0]
24 df_cleaned = df_cleaned[df_cleaned['UnitPrice'] > 0]
25 df_cleaned['CustomerID'] = df_cleaned['CustomerID'].astype(int)
26 print(f"Veri temizlendi. Analiz için kalan satır sayısı: {len(df_cleaned)}")
27
28 # --- 2. RFM METRİKLERİNİ HESAPLAMA ---
29 print("RFM metrikleri hesaplanıyor...")
30 df_cleaned['TotalPrice'] = df_cleaned['Quantity'] * df_cleaned['UnitPrice']
31 df_cleaned['InvoiceDate'] = pd.to_datetime(df_cleaned['InvoiceDate'], format='%d-%m-%Y %H:%M')
32 snapshot_date = df_cleaned['InvoiceDate'].max() + dt.timedelta(days=1)
33 rfm_df = df_cleaned.groupby('CustomerID').agg(
34     Recency=('InvoiceDate', lambda x: (snapshot_date - x.max()).days),
35     Frequency=('InvoiceNo', 'nunique'),
36     Monetary=('TotalPrice', 'sum')
37 ).reset_index()
38
39 # --- 3. VERİYİ ÖLÇEKLENDİRME (SCALING) ---
40 print("RFM verisi ölçeklendiriliyor (StandardScaler)...")
41 rfm_features = rfm_df[['Recency', 'Frequency', 'Monetary']]
42 scaler = StandardScaler()
43 rfm_scaled = scaler.fit_transform(rfm_features)
44
45 # --- 4. AFFINITY PROPAGATION MODELİNİ UYGULAMA ---
46 print("\nAffinity Propagation modeli çalıştırılıyor...")
47 print("UYARI: Bu işlem 5-10+ dakika sürebilir. Lütfen bekleyin...")
48
49 start_time = time.time()
50 model_ap = AffinityPropagation(damping=0.9, random_state=42)
51 model_ap.fit(rfm_scaled)
52 duration = time.time() - start_time
53 labels = model_ap.labels_
54 rfm_df['Cluster'] = labels
55
56 print("Kümeleme tamamlandı.")
57
58 # --- 5. AFFINITY PROPAGATION PERFORMANS METRİKLERİ ---
59 print("\n--- Affinity Propagation Performans Metrikleri ---")
60 print(f"Model Çalışma Süresi: {duration:.2f} saniye")
61
62 # Bulunan küme sayısını kontrol et
63 n_clusters_found = len(np.unique(labels))
64 print(f"Bulunan Küme Sayısı: {n_clusters_found}")
65
66 if n_clusters_found < 2:
67     print("Metrik hesaplanamıyor (1 veya daha az küme bulundu).")
68 else:
69     silhouette = silhouette_score(rfm_scaled, labels)
70     davies_bouldin = davies_bouldin_score(rfm_scaled, labels)
71     calinski_harabasz = calinski_harabasz_score(rfm_scaled, labels)
72
73     print(f"Siluet Skoru (İyisi +1'e yakın): {silhouette:.3f}")
74     print(f"Davies-Bouldin Endeksi (İyisi 0'a yakın): {davies_bouldin:.3f}")
75     print(f"Calinski-Harabasz Endeksi (İyisi yüksek): {calinski_harabasz:.3f}")
76
77 # --- 6. KÜME SONUÇLARINI ANALİZ ETME ---
78 print("\n--- Küme Analizi (Ortalama RFM Değerleri) ---")
79 cluster_summary_ap = rfm_df.groupby('Cluster').agg(
80     Recency_Avg=('Recency', 'mean'),
81     Frequency_Avg=('Frequency', 'mean'),
82     Monetary_Avg=('Monetary', 'mean'),
83     Count=('CustomerID', 'count')
84 ).sort_values('Count', ascending=False)
85 print(cluster_summary_ap)
86
87 rfm_df.to_csv('customer_segments_ap.csv', index=False)
88 print("\nSonuçlar 'customer_segments_ap.csv' dosyasına kaydedildi.")
89 print("Script tamamlandı.")

```

Affinity Propagation Script'i başlatıldı...

Veri temizlendi. Analiz için kalan satır sayısı: 397884

RFM metrikleri hesaplanıyor...

RFM verisi ölçeklendiriliyor (StandardScaler)...

Affinity Propagation modeli çalıştırılıyor...

UYARI: Bu işlem 5-10+ dakika sürebilir. Lütfen bekleyin...

Kümeleme tamamlandı.

--- Affinity Propagation Performans Metrikleri ---

Model Çalışma Süresi: 24.10 saniye

Bulunan Küme Sayısı: 96

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Davies-Bouldin Endeksi (İyisi 0'a yakın): 0.676

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Siluet Skoru (İyisi +1'e yakın): 0.341

Davies-Bouldin Endeksi (İyisi 0'a yakın): 0.676

Calinski-Harabasz Endeksi (İyisi yüksek): 7627.161

--- Küme Analizi (Ortalama RFM Değerleri) ---

	Recency_Avg	Frequency_Avg	Monetary_Avg	Count
Cluster				
23	12.52	2.00	592.54	219
87	56.69	1.03	383.44	195
64	33.05	1.00	324.08	174
13	12.48	4.00	1147.14	155
42	29.04	3.16	919.71	146
...	...	...	...	...
83	8.00	46.00	194550.79	1
85	3.00	31.00	91062.38	1
89	2.00	124.00	40991.57	1
90	372.00	34.00	5391.21	1
93	1.00	60.00	259657.30	1

[96 rows x 4 columns]

Sonuçlar 'customer_segments_ap.csv' dosyasına kaydedildi.
Script tamamlandı.

```

mean_shift.py >...
1 import pandas as pd
2 import datetime as dt
3 import warnings
4 import time # Süre ölçümü için
5 import numpy as np # Küme sayısı kontrolü ve örnekleme için
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.cluster import MeanShift, estimate_bandwidth
8 from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
9
10 # --- Genel Ayarlar ---
11 pd.set_option('display.float_format', lambda x: '%.2f' % x)
12 warnings.filterwarnings('ignore')
13 print("Mean Shift Script'i başlatıldı...")
14
15 # --- 1. VERİ YÜKLEME VE TEMİZLEME ---
16 try:
17     df = pd.read_csv('OnlineRetail.csv', encoding='ISO-8859-1')
18 except FileNotFoundError:
19     print("HATA: 'OnlineRetail.csv' dosyası bulunamadı.")
20     exit()
21
22 df_cleaned = df.dropna(subset=['CustomerID'])
23 df_cleaned = df_cleaned[df_cleaned['Quantity'] > 0]
24 df_cleaned = df_cleaned[df_cleaned['UnitPrice'] > 0]
25 df_cleaned['CustomerID'] = df_cleaned['CustomerID'].astype(int)
26 print(f"Veri temizlendi. Analiz için kalan satır sayısı: {len(df_cleaned)}")
27
28 # --- 2. RFM METRİKLERİNİ HESAPLAMA ---
29 print("RFM metrikleri hesaplanıyor...")
30 df_cleaned['TotalPrice'] = df_cleaned['Quantity'] * df_cleaned['UnitPrice']
31 df_cleaned['InvoiceDate'] = pd.to_datetime(df_cleaned['InvoiceDate'], format='%d-%m-%Y %H:%M')
32 snapshot_date = df_cleaned['InvoiceDate'].max() + dt.timedelta(days=1)
33 rfm_df = df_cleaned.groupby('CustomerID').agg(
34     Recency=('InvoiceDate', lambda x: (snapshot_date - x.max()).days),
35     Frequency=('InvoiceNo', 'nunique'),
36     Monetary=('TotalPrice', 'sum')
37 ).reset_index()
38
39 # --- 3. VERİYİ ÖLÇEKLENDİRME (SCALING) ---
40 print("RFM verisi ölçeklendiriliyor (StandardScaler)...")
41 rfm_features = rfm_df[['Recency', 'Frequency', 'Monetary']]
42 scaler = StandardScaler()
43 rfm_scaled = scaler.fit_transform(rfm_features)
44
45 # --- 4. MEAN SHIFT MODELİNİ UYGULAMA ---
46 print("\nMean Shift modeli için 'bandwidth' (bant genişliği) tahmin ediliyor...")
47 print("UYARI: Bu işlem 1-2 dakika sürebilir...")
48 try:
49     # İşlemi hızlandırmak için 1000 rastgele örnek al
50     n_samples_for_bandwidth = min(1000, len(rfm_scaled))
51     indices = np.random.choice(rfm_scaled.shape[0], n_samples_for_bandwidth, replace=False)
52     bandwidth_data = rfm_scaled[indices]
53     bandwidth = estimate_bandwidth(bandwidth_data, quantile=0.2, n_jobs=-1)
54     if bandwidth <= 0.01: bandwidth = 1.0 # Çok küçükse varsayılan ata
55     print(f"Tahmin edilen optimal bandwidth: {bandwidth:.3f}")
56 except Exception:
57     bandwidth = 1.0
58     print(f"Bandwidth tahmini başarısız. Varsayılan (1.0) kullanılıyor.")
59
60 print("Mean Shift modeli çalıştırılıyor (Bu da zaman alabilir)...")
61 start_time = time.time()
62 model_ms = MeanShift(bandwidth=bandwidth, bin_seeding=True, n_jobs=-1)
63 model_ms.fit(rfm_scaled)
64 duration = time.time() - start_time
65 labels = model_ms.labels_
66 rfm_df['Cluster'] = labels
67
68 print("Kümeleme tamamlandı.")
69
70 # --- 5. MEAN SHIFT PERFORMANS METRİKLERİ ---
71 print("\n--- Mean Shift Performans Metrikleri ---")
72 print(f"Model Çalışma Süresi: {duration:.2f} saniye")
73
74 # Bulunan küme sayısını kontrol et
75 n_clusters_found = len(np.unique(labels))
76 print(f"Bulunan Küme Sayısı: {n_clusters_found}")
77
78 if n_clusters_found < 2:
79     print("Metrik hesaplanıyor (1 veya daha az küme bulundu).")
80 else:
81     silhouette = silhouette_score(rfm_scaled, labels)
82     davies_bouldin = davies_bouldin_score(rfm_scaled, labels)
83     calinski_harabasz = calinski_harabasz_score(rfm_scaled, labels)
84
85     print(f"Siluet Skoru (İyisi +1'e yakın): {silhouette:.3f}")
86     print(f"Davies-Bouldin Endeksi (İyisi 0'a yakın): {davies_bouldin:.3f}")
87     print(f"Calinski-Harabasz Endeksi (İyisi yüksek): {calinski_harabasz:.3f}")
88
89 # --- 6. KÜME SONUÇLARINI ANALİZ ETME ---
90 print("\n--- Küme Analizi (Ortalama RFM Değerleri) ---")
91 cluster_summary_ms = rfm_df.groupby('Cluster').agg(
92     Recency_Avg=('Recency', 'mean'),
93     Frequency_Avg=('Frequency', 'mean'),
94     Monetary_Avg=('Monetary', 'mean'),
95     Count=('CustomerID', 'count')
96 ).sort_values('Count', ascending=False)
97 print(cluster_summary_ms)
98
99 rfm_df.to_csv('customer_segments_ms.csv', index=False)
100 print("\nSonuçlar 'customer_segments_ms.csv' dosyasına kaydedildi.")
101 print("Script tamamlandı.")

```

Mean Shift Script'i başlatıldı...
Veri temizlendi. Analiz için kalan satır sayısı: 397884
RFM metrikleri hesaplanıyor...
RFM verisi ölçeklendiriliyor (StandardScaler)...

Mean Shift modeli için 'bandwidth' (bant genişliği) tahmin ediliyor...
UYARI: Bu işlem 1-2 dakika sürebilir...
Tahmin edilen optimal bandwidth: 0.775
Mean Shift modeli çalıştırılıyor (Bu da zaman alabilir)...
Kümeleme tamamlandı.

--- Mean Shift Performans Metrikleri ---
Model Çalışma Süresi: 3.20 saniye
Bulunan Küme Sayısı: 35
Siluet Skoru (İyisi +1'e yakın): 0.376
Davies-Bouldin Endeksi (İyisi 0'a yakın): 0.309
Calinski-Harabasz Endeksi (İyisi yüksek): 204.821

--- Küme Analizi (Ortalama RFM Değerleri) ---

Cluster	Recency_Avg	Frequency_Avg	Monetary_Avg	Count
0	95.89	3.26	1145.62	4156
1	9.12	21.31	9112.55	90
10	19.82	10.14	13429.23	28
2	7.38	31.94	5984.25	16
3	5.00	36.40	16243.25	5
4	9.75	19.75	25836.56	4
5	12.33	16.33	35868.62	3
6	7.33	51.67	36322.95	3
7	4.33	45.67	54857.31	3
8	1.00	31.67	28897.76	3
9	208.50	3.00	42225.40	2
11	2.00	94.00	59796.87	2
12	372.00	34.00	5391.21	1
13	326.00	1.00	77183.60	1
14	39.00	63.00	81024.84	1
15	24.00	21.00	124914.53	1
16	10.00	55.00	117379.63	1
17	10.00	47.00	19914.44	1
18	10.00	13.00	50491.81	1
19	8.00	46.00	194550.79	1
20	7.00	26.00	56252.72	1
21	4.00	86.00	11189.91	1
22	4.00	50.00	65039.62	1
23	4.00	28.00	66653.56	1
24	4.00	17.00	65164.79	1
25	3.00	55.00	8508.82	1
26	3.00	31.00	91062.38	1
27	2.00	124.00	40991.57	1
28	2.00	73.00	280206.02	1
29	2.00	62.00	28117.04	1
30	1.00	209.00	33719.73	1
31	1.00	201.00	143825.06	1
32	1.00	93.00	12156.65	1
33	1.00	60.00	259657.30	1
34	1.00	2.00	168472.50	1

Sonuçlar 'customer_segments_ms.csv' dosyasına kaydedildi.
Script tamamlandı.